Recognising without recalling:

a small recurrent memory recognises long after it stops recalling

Ivan Rodkin

benchmark: `irodkin/recognition-memory`

Takeaway

A memory can tell whether an input is familiar long after it can no longer reproduce it.

- Shrinking a GDN-2 state, **exact recall fails first**: at 16×16, $N{=}256$ nothing is copied exactly, yet both readouts score acc9 ≥ 0.996.
- At the smallest state (16×4) the memory **trained for recognition** wins in **3 of 3 runs**: acc9 **0.86** [0.78–0.98] vs. **0.45** [0.27–0.63] for surprise at $N{=}256$ (chance 0.11).
- Training it needs a **length curriculum**.

Hypothesis

Memory need not *reproduce* the past. It can instead *recognise* whether an input is consistent with it, using low **surprise** as the familiarity signal. Checking is easier than generating, so recognition should need **less capacity** than recall.

Benchmark: one sequence, eight near-copies

Uniform i.i.d. tokens, vocabulary 16 (4 bits each), $N = 4 \dots 256$. Each comes with **8 corrupted copies**: one forced change plus $p = \frac{1}{N-1}$ elsewhere, so **2 changes on average**.

true	3	a	0	f	7	7	c	1	9	e	4	2	b	5	d	8
copy	3	a	0	f	7	**2**	c	1	9	e	4	2	b	5	d	8

Task. Score the true sequence and its 8 copies.
acc9: true one ranked first (chance $1/9$). **pairwise**: true beats one copy (chance $1/2$).

Capacity, in theory. Recall needs all $4N$ bits (1,024 at $N{=}256$). Recognition needs a few: keeping only the token sum mod 16 (4 bits) gives acc9 0.85; adding a position-weighted sum (8 bits) gives 0.965, at *every* N.

Model: one network, two readouts

2-layer Gated DeltaNet-2 (`fla`), width 128, one head. The only memory is the recurrent state, $d_k \times d_v$ per layer: 128×128 (16,384 numbers) down to 16×4 (64).

state $S_t \in \mathbb{R}^{d_k \times d_v}$ carries x forward

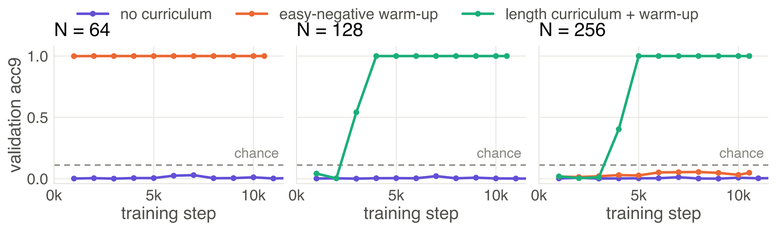
BOS | **sequence** $x_{1..N}$ | **SEP** | **candidate** $c_{1..N}$

Recall model
trained to copy x (next-token CE)
score $=$ $-$surprise
$= \sum_j \log p(c_j \mid x, c_{<j})$

Recognition model
trained on "is $c = x$?"
(BCE, or ranking loss: block 2)
score $=$ logit at the last token

1 Full-size state: recall is easy, recognition needs a curriculum

At 128×128 the **recall model** copies every length almost perfectly (exact copy 0.988 at $N{=}256$), and its surprise ranks the true sequence first in *every* test sample. The **recognition model**, trained directly, fails from $N{=}64$ on: its loss sits at $\ln 2$ (a constant output) for all 35k steps. The same network learns recall in $<1{,}000$ steps, so this is an **optimisation** failure, not a capacity limit.
Length curriculum: for the first half of training, use prefixes $16, 32, \dots, N$ (equal time each), plus an **easy-negative warm-up** (heavily corrupted negatives that fade out); then full length.

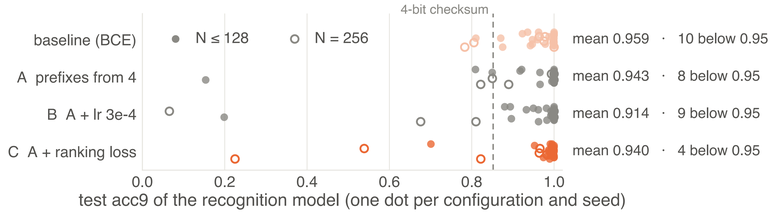


Validation acc9 (full-length sequences), 128×128 recognition model. Curriculum runs: 3 epochs; no-curriculum runs: 10 epochs (first 11k steps shown).

- **Length curriculum**: acc9 $= 1.000$ at $N{=}128$ and 256. The warm-up alone fixes only $N{=}64$.
- With it, **all 72** recognition runs of the sweep and seeds train; none ends at $\ln 2$.

2 A ranking loss? A hypothesis, not a fix

Some small-state runs stall below their capacity. Three fixes were tried on the 11 configurations (16×4–16×16) where a baseline run fell short, seeds 0–2. Fix **C** trains the same last-token score on (true, corrupted) pairs: loss $= \mathrm{softplus}(s_{\text{neg}} - s_{\text{true}})$, i.e. *rank*, don't calibrate.

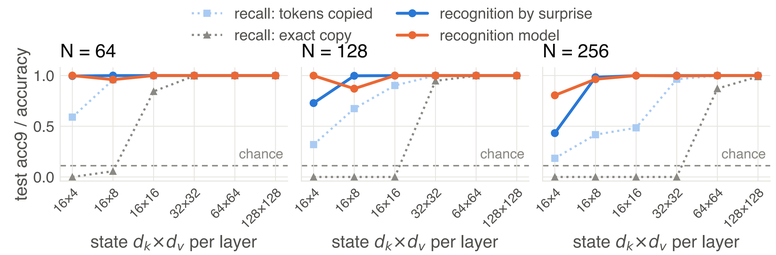


33 runs per version. Same seed numbers, but *not* paired runs.

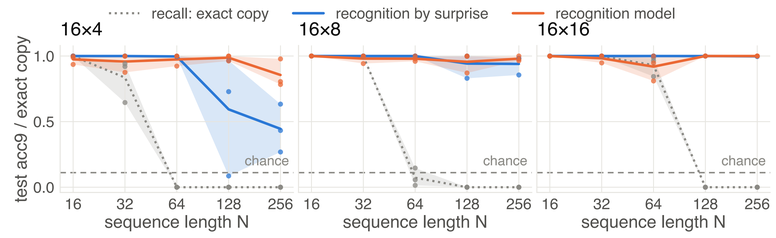
- **No fix beats the baseline on average** (**C** 0.940 vs. 0.959).
- Up to $N{=}128$, **C** has 1 run below 0.95 against 8; but that cut was made after seeing the results, and configurations picked for a weak baseline run favour *any* fix (regression to the mean).
- At $N{=}256$, 16×4, **C** is worse: 0.58 vs. 0.86 (slow runs, not stuck).
- Fresh seeds for both, and longer training, are running.

3 Shrinking the state: recall goes first

6 state sizes (16×4 to 128×128) $\times$ 7 lengths $\times$ 2 objectives, 10 epochs each, tested on 100k sequences; 3 seeds at the three smallest states. Recognition model: BCE + curriculum.



Seed 0, test acc9: **recall model's surprise**, **recognition model**. Dotted: recall. $N \leq 32$: all ≥ 0.87.



Three smallest states, 3 seeds: mean (line), min–max (band), each seed (dots).

- **Recognition outlasts recall.** At 16×16, $N{=}256$ the recall model keeps ~half the bits (≥ 536 of 1,024) and copies nothing exactly, yet both readouts score acc9 ≥ 0.996.
- **Smallest state: the recognition model wins in 3 of 3 runs.**

16×4, 3 seeds	recall tokens	surprise acc9	recognition acc9
$N{=}128$	0.30	0.59 [0.09–0.97]	**0.99** [0.96–1.00]
$N{=}256$	0.20	0.45 [0.27–0.63]	**0.86** [0.78–0.98]

  Three runs per cell is little evidence, but the margins are large. On average level with a 4-bit checksum (0.85); its best run (0.98) beats the 8-bit one (0.965).
- Near capacity, **surprise** depends on what the memory chose to keep: 0.97, 0.73, 0.09 across seeds at $N{=}128$.
- At 16×8 the two are about even ($N{=}256$: 0.94 vs. 0.98).

Next

- **Fresh seeds** (running): is any training version reliable at the smallest states? If so, rerun the small-state sweep with it.
- **Harder benchmarks** where checksums fail; K stored sequences per context.

3 seeds at 16×4–16×16 for $N \geq 16$, seed 0 elsewhere. The three smallest states shrink d_v only (fla kernels need $d_k \geq 16$); across them the parameter count changes by 4%.